LAB DIGITAL REPORT

Student Enrolment Number	**A86904824018**
Name of the Student	**Madiha Mufassira**
Programme	**BCA**
Course Code	**CSE1021**
Course Title	**Python programming**
Semester	**Fall**
Name of the Faculty	**Dr.Vigneesh Ramamoorthy**

INDEX

S.NO	*DATE*	*EXPERIMENT TITLE*
1	19-09-2025	VARIABLES & DATA TYPES
2	19-09-2025	OPERATORS
3	22-09-2025	TYPE CONVERSION
4	22-09-2025	BASIC I/O
5	26-09-2025	MATH FUNCTIONS
6	26-09-2025	CONSTANTS REASSIGNMENT
7	29-09-2025	TYPE CHECKING
8	29-09-2025	FORMATTED OUTPUT

9	10-10-2025	CONDITIONALS
10	10-10-2025	FOR LOOPS
11	13-10-2025	WHILE LOOPS
12	13-10-2025	LOOP CONTROL
13	17-10-2025	PASS STATEMENT
14	17-10-2025	STRING SLICING
15	24-10-2025	STRING METHODS
16	24-10-2025	FUNCTIONS
17	27-10-2025	FUNCTION PARAMETERS
18	27-10-2025	RETURN VALUES
19	31-10-2025	LAMBDA FUNCTION
20	31-10-2025	RECURSION
21	14-11-2025	VARIABLE SCOPE
22	14-11-2025	LIST OPERATIONS

23	17-11-2025	LIST COMPREHENSION
24	21-11-2025	TUPLES
25	24-11-2025	SETS
26	28-11-2025	DICTIONARIES
27	01-12-2025	ARRAYS
28	05-12-2025	EXCEPTION HANDLING
29	08-12-2025	CLASS OBJECT
30	12-12-2025	INHERITANCE
31	29-12-2025	ENCAPSULATION
32	02-01-2026	POLYMORPHISM
33	05-01-2026	ABSTRACTION
34	09-01-2026	MODULES
35	10-01-2026	RANDOM MODULE

EX.NO: 1

VARIABLES AND DATA TYPES

CODE:

```
num1=22

num1=num1+1

num1

num1=num1*5

num1

num2=num1

num1

num2
```

OUTPUT:

```
Output                                  Clear
num1=22
num1=num1+1
num1
23
num1=num1*5
num1
115
num2=num1
num1
115
num2
115
=== Code Execution Successful ===
```

EX.NO: 2

PROGRAM USING OPERATORS.

CODE:

```
#RELATIONAL OPERATOR
x=10
y=5
x=y
print(x)
x==y
z=20
x==z
x!=y
z!=y
z>y
x
x>y
y<z
x<=y
y<=x
x>=y
#LOGICAL OPERATOR
(x>y) and (x==y)
(z>y) and (x==y)
(x>y) or (x==y)
(z>y) or (x==y)
a=x==y
print(a)
a=(x==y)
print(a)
p=not(a)
print(p)
q=(z<x)
print(q)
s=not(q)
print(s)
```

OUTPUT:

Output

Clear

```
5
True
True
False
False
True

=== Code Execution Successful ===
```

EX.NO: 3

PROGRAM USING TYPE CONVERSION.

CODE:

```
print("----- IMPLICIT TYPE CONVERSION -----")
# int to float
a = 10
b = 2.5
c = a + b
print("a =", a, type(a))
print("b =", b, type(b))
print("c = a + b =", c, type(c))  # int → float

print("\n----- EXPLICIT TYPE CONVERSION -----")
# int ↔ float
x = 15
y = float(x)
z = int(9.8)
print("int to float:", y, type(y))
print("float to int:", z, type(z))

print("\n----- STRING CONVERSION -----")
num = 100
s = str(num)
print("int to string:", s, type(s))
s2 = "200"
n2 = int(s2)
print("string to int:", n2, type(n2))
s3 = "3.14"
f3 = float(s3)
print("string to float:", f3, type(f3))
```

```
print("\n----- BOOLEAN CONVERSION -----")
print("bool(0):", bool(0))
print("bool(1):", bool(1))
print("bool(100):", bool(100))
print("bool(''):", bool(""))
print("bool('Python'):", bool("Python"))

print("\n----- COMPLEX NUMBER CONVERSION -----")
r = 5
c = complex(r)
print("int to complex:", c, type(c))
f = 2.5
c2 = complex(f)
print("float to complex:", c2, type(c2))

print("\n----- COLLECTION CONVERSION -----")
# List, Tuple, Set
lst = [1, 2, 3, 3]
tpl
```

OUTPUT:

```
----- IMPLICIT TYPE CONVERSION -----
a = 10 <class 'int'>
b = 2.5 <class 'float'>
c = a + b = 12.5 <class 'float'>

----- EXPLICIT TYPE CONVERSION -----
int to float: 15.0 <class 'float'>
float to int: 9 <class 'int'>

----- STRING CONVERSION -----
int to string: 100 <class 'str'>
string to int: 200 <class 'int'>
string to float: 3.14 <class 'float'>

----- BOOLEAN CONVERSION -----
bool(0): False
bool(1): True
bool(100): True
bool(''): False
bool('Python'): True

----- COMPLEX NUMBER CONVERSION -----
ERROR!
int to complex: (5+0j) <class 'complex'>
float to complex: (2.5+0j) <class 'complex'>

----- COLLECTION CONVERSION -----
Traceback (most recent call last):
  File "<main.py>", line 47, in <module>
NameError: name 'tpl' is not defined

=== Code Exited With Errors ===
```

EX.NO: 4

PROGRAM FOR BASIC INPUT/OUTPUT.

CODE:

```
print("Welcome to Python Input-Output Program")
# Printing multiple values
a = 10
b = 20
print("a =", a, "b =", b)

# Using sep and end
print("Python", "Java", "C++", sep=" | ", end=" ---> ")
print("Programming Languages")

# ----- INPUT USING input() -----
name = input("\nEnter your name: ")

# input with type conversion
age = int(input("Enter your age: "))
height = float(input("Enter your height: "))

# ----- MULTIPLE INPUTS IN ONE LINE -----
x, y = map(int, input("Enter two integers: ").split())

# ----- LIST INPUT -----
numbers = list(map(int, input("Enter list elements: ").split()))

# ----- BOOLEAN CONVERSION -----
status = input("Are you a student (True/False): ")
status = status == "True"

# ----- OUTPUT FORMATTING -----
```

```
print("\n----- USER DETAILS -----")
print(f"Name: {name}")
print("Age:", age)
print("Height:", height)

print("Two numbers:", x, y)
print("Sum:", x + y)

print("List:", numbers)
print("Student Status:", status)

# ----- SPECIAL CHARACTERS -----
print("\nSpecial Characters Demo")
print("Hello\nWorld")
print("Hello\tPython")

# ----- TYPE CHECKING -----
print("\nType Checking")
print("Type of age:", type(age))
print("Type of height:", type(height))
print("Type of numbers:", type(numbers))

print("\nProgram Completed Successfully")
```

OUTPUT:

```
Welcome to Python Input-Output Program
a = 10 b = 20
Python | Java | C++ ---> Programming Languages

Enter your name: madiha
Enter your age: 19
Enter your height: 5.7
Enter two integers: 2 8
Enter list elements: 6
Are you a student (True/False): True

----- USER DETAILS -----
Name: madiha
Age: 19
Height: 5.7
Two numbers: 2 8
Sum: 10
List: [6]
Student Status: True

Special Characters Demo
Hello
World
Hello   Python

Type Checking
Type of age: <class 'int'>
Type of height: <class 'float'>
Type of numbers: <class 'list'>

Program Completed Successfully

=== Code Execution Successful ===
```

EX.NO: 5

Program using math library functions – sqrt, power, ceil, floor.

CODE:

```
import math
num=float(input("enter the number:-"))
square_root=math.sqrt(num)
square_root2=math.sqrt(36)
print(square_root)
print(square_root2)
square_root3=math.sqrt(2025)
print(square_root3)
power_value=math.pow(4,3)
print(power_value)
power_value2=math.pow(4,3)
print(power_value)
power_value2=math.pow(8,6)
print(power_value)
print(power_value2)
a=float(input("yellu geleya"))
ciel_value=math.ceil(a)
a
b=math.ceil(a)
b
c=math.ceil(a)
c
```

OUTPUT:

```
Output

enter the number:-28
5.291502622129181
6.0
45.0
64.0
64.0
64.0
262144.0
yellu geleya
```

EX.NO: 6

Program with constants, variables and demonstrate reassignment.

CODE:

```
# CONSTANTS (by convention)
PI = 3.14159
MAX_SCORE = 100
print("Constants:")
print("PI =", PI)
print("MAX_SCORE =", MAX_SCORE)

# VARIABLES
x = 10          # integer variable
y = 5.5         # float variable
name = "Alice"  # string variable
print("\nVariables:")
print("x =", x)
print("y =", y)
print("name =", name)

# REASSIGNMENT
x = 20          # reassigned value
name = "Bob"    # reassigned value
print("\nAfter Reassignment:")
print("x =", x)
print("name =", name)

# DYNAMIC TYPING
value = 100
print("\nValue:", value, type(value))
value = "Python"
print("Value:", value, type(value))
```

```
# USING CONSTANT IN CALCULATION
radius = 5
area = PI * radius * radius
print("\nArea of circle =", area)
```

OUTPUT:

```
Output                                    Clear
Constants:
PI = 3.14159
MAX_SCORE = 100

Variables:
x = 10
y = 5.5
name = Alice

After Reassignment:
x = 20
name = Bob

Value: 100 <class 'int'>
Value: Python <class 'str'>

Area of circle = 78.53975

=== Code Execution Successful ===
```

EX.NO: 7

PROGRAM TO CHECK TYPE OF OBJECT USING TYPE() AND ISINSTANCE().

CODE:

```
# ----- BASIC TYPE CHECKING USING type() ---
a = 10
b = 5.5
c = "Python"
d = True
e = [1, 2, 3]
f = (4, 5, 6)
g = {7, 8, 9}
h = {"name": "Alice", "age": 20}
print("Basic Type Checking:")
print("a =", a, "->", type(a))
print("b =", b, "->", type(b))
print("c =", c, "->", type(c))
print("d =", d, "->", type(d))
print("e =", e, "->", type(e))
print("f =", f, "->", type(f))
print("g =", g, "->", type(g))
print("h =", h, "->", type(h))
# ----- TYPE CHECKING USING isinstance() -----
print("\nUsing isinstance():")
print("a is int:", isinstance(a, int))
print("b is float:", isinstance(b, float))
print("c is str:", isinstance(c, str))
print("d is bool:", isinstance(d, bool))
print("e is list:", isinstance(e, list))
print("f is tuple:", isinstance(f, tuple))
print("g is set:", isinstance(g, set))
print("h is dict:", isinstance(h, dict))
```

```
# ----- CHECKING MULTIPLE TYPES -----
print("\nChecking Multiple Types:")
x = 10.5
print("x is int or float:", isinstance(x, (int, float)))
# ----- USER INPUT TYPE CHECKING -----
value = input("\nEnter any value: ")
print("Input value:", value)
print("Type before conversion:", type(value))
num = int(value)
print("After conversion to int:", num)
print("Type after conversion:", type(num))
# ----- DYNAMIC TYPING -----
print("\nDynamic Typing Example:")
var = 100
print(var, type(var))
var = "Hello"
print(var, type(var))
var = 3.14
print(var, type(var))
# ----- BOOLEAN TYPE CHECKING -----
print("\nBoolean Type Checking:")
print("bool(0):", bool(0), type(bool(0)))
print("bool(1):", bool(1), type(bool(1)))
print("bool(''):", bool(""), type(bool("")))
print("bool('Python'):", bool("Python"), type(bool("Python")))
# ----- NONE TYPE CHECKING -----
print("\nNone Type Checking:")
value = None
print(value, type(value))
print("Is value None:", value is None)
print("\nProgram Completed Successfully")
```

OUTPUT:

Output

```
Basic Type Checking:
a = 10 -> <class 'int'>
b = 5.5 -> <class 'float'>
c = Python -> <class 'str'>
d = True -> <class 'bool'>
e = [1, 2, 3] -> <class 'list'>
f = (4, 5, 6) -> <class 'tuple'>
g = {8, 9, 7} -> <class 'set'>
h = {'name': 'Alice', 'age': 20} -> <class 'dict'>

Using isinstance():
a is int: True
b is float: True
c is str: True
d is bool: True
e is list: True
f is tuple: True
g is set: True
h is dict: True

Checking Multiple Types:
x is int or float: True

Enter any value: 20
Input value: 20
Type before conversion: <class 'str'>
After conversion to int: 20
Type after conversion: <class 'int'>

Dynamic Typing Example:
100 <class 'int'>
Hello <class 'str'>
3.14 <class 'float'>

Boolean Type Checking:
bool(0): False <class 'bool'>
bool(1): True <class 'bool'>
bool(''): False <class 'bool'>
bool('Python'): True <class 'bool'>

None Type Checking:
None <class 'NoneType'>
Is value None: True

Program Completed Successfully
```

EX.NO: 8

J

```
name = "Madiha mufassira"
age = 19
marks = 95.6
print("Name:", name, "Age:", age)

#Using % (Old-style formatting)
print("Name: %s Age: %d Marks: %.2f" % (name, age, marks))

#Using str.format() method
print("Name: {} Age: {} Marks: {:.2f}".format(name, age, marks))

#Using f-strings (Recommended – Python 3.6+)
print(f"Name: {name} Age: {age} Marks: {marks:.2f}")

#Formatting numbers
#Decimal places
pi = 3.1415926
print(f"Pi value: {pi:.3f}")

#Width & alignment
print(f"{'Name':<10}{'Marks':>10}")
print(f"{'madiha':<10}{85:>10}")

#Formatting tables
print(f"{'ID':<5}{'Name':<10}{'Score':<5}")
print(f"{1:<5}{'madiha':<10}{90:<5}")
print(f"{2:<5}{'sneha':<10}{88:<5}")

#Formatting dates
```

```
from datetime import date
today = date.today()

print(today.strftime("%d-%m-%Y"))
```

OUTPUT:

```
main.py    Output
Name: Madiha mufassira Age: 19
Name: Madiha mufassira Age: 19 Marks: 95.60
Name: Madiha mufassira Age: 19 Marks: 95.60
Name: Madiha mufassira Age: 19 Marks: 95.60
Pi value: 3.142
Name           Marks
madiha            85
ID   Name      Score
1    madiha    90
2    sneha     88
13-01-2026

=== Code Execution Successful ===
```

EX.NO: 9

PROGRAM COMBINING ALL IF-TYPES – IF, IF–ELSE, IF–ELIF–ELSE, NESTED IF.

CODE:

```
#if Statement
age = 18
if age >= 18:
    print("You are eligible to vote")
#if – else Statement
num = 5
if num % 2 == 0:
    print("Even number")
else:
    print("Odd number")
#if – elif – else Statement
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
#Nested if Statement
num = 10
if num > 0:
    if num % 2 == 0:
        print("Positive Even number")
    else:
        print("Positive Odd number")
else:
    print("Negative number")
```

```
#Conditional (Ternary) Expression
a = 10
b = 20
max_value = a if a > b else b
print("Maximum value:", max_value)
#Using Logical Operators in Conditionals
username = "madiha"
password = "1234"
if username == "madiha" and password == "1234":
    print("Login successful")
else:
    print("Invalid credentials")
#if with Membership Operators
fruits = ["apple", "banana", "mango"]
if "apple" in fruits:
    print("Apple is available")
```

OUTPUT:

```
main.py    Output
You are eligible to vote
Odd number
Grade B
Positive Even number
Maximum value: 20
Login successful
Apple is available

=== Code Execution Successful ===
```

EX.NO: 10

PROGRAM WITH FOR LOOP, FOR WITH ELSE BLOCK, NESTED FOR.

CODE:

```
#Loop through a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
#Using range()
for i in range(5):
    print(i)
#Loop with start, stop, step
for i in range(1, 10, 2):
    print(i)
#Loop through a string
for char in "Python":
    print(char)
#Using break
for i in range(1, 6):
    if i == 4:
        break
    print(i)
#Using continue
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
# Nested for loop
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j)
#For loop with else
for i in range(5):
```

```
    print(i)
else:
    print("Loop completed successfully")
```

OUTPUT:

```
Output
apple
banana
cherry
0
1
2
3
4
1
3
5
7
9
p
y
t
h
o
n
1
2
3
1
2
4
5
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
0
1
2
3
4
Loop completed successfully

=== Code Execution Successful ===
```

EX.NO: 11

PROGRAM WITH WHILE LOOP AND WHILE…ELSE.

CODE:

```
#Print numbers from 1 to 5
i = 1
while i <= 5:
    print(i)
    i += 1
#Sum of first 10 numbers
i = 1
total = 0

while i <= 10:
    total += i
    i += 1

print("Sum =", total)
#Using break in while loop
i = 1
while i <= 10:
    if i == 6:
        break
    print(i)
    i += 1
#Using continue in while loop
i = 0
while i < 5:
    i += 1
    if i == 3:
        continue
    print(i)
#while loop with else
```

```
i = 1
while i <= 3:
    print(i)
    i += 1
else:
    print("Loop completed")
#Infinite while loop
while True:
    print("Hello")
```

OUTPUT:

```
Output                                  Clear
1
2
3
4
5
Sum = 55
1
2
3
4
5
1
2
4
5
1
2
3
Loop completed
Hello
Hello
Hello
Hello
Hello
Hello
```

```
Output                                  Clear
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello

=== Execution Halted ===
```

EX.NO: 12

PROGRAM WITH LOOP CONTROL – BREAK AND CONTINUE.

CODE:

```
for i in range(1, 6):
    if i == 4:
        break
    print(i)

#continue Statement
for i in range(1, 6):
    if i == 3:
        continue
    print(i)

#pass Statement
for i in range(1, 6):
    if i == 3:
        pass
    print(i)

#else with Loops
for i in range(1, 4):
    print(i)
else:
    print("Loop completed")

#Example (with break):
for i in range(1, 4):
    if i == 2:
        break
    print(i)
else:
```

```
print("Loop completed")
```

OUTPUT:

```
Output                                  Clear
1
2
3
1
2
4
5
1
2
3
4
5
1
2
3
Loop completed
1

=== Code Execution Successful ===
```

EX.NO: 13

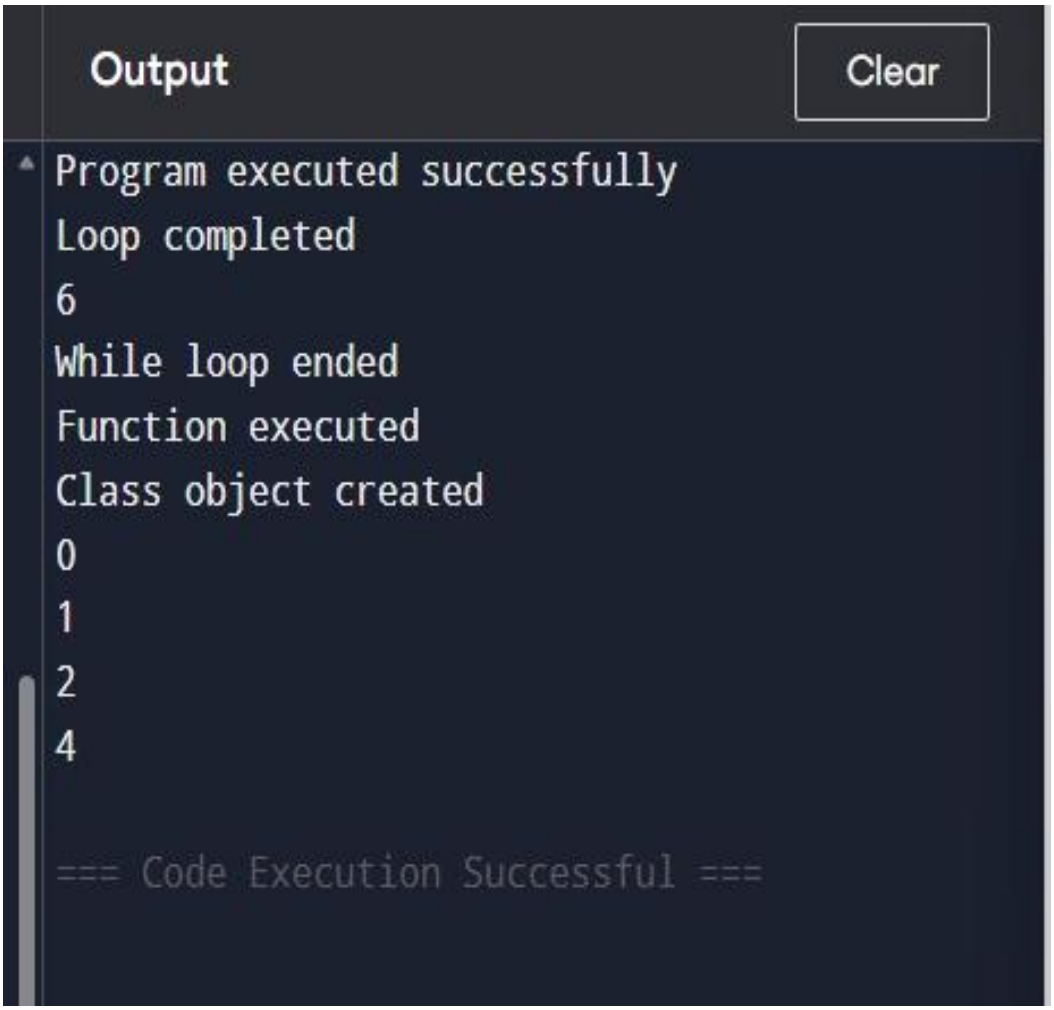
PROGRAM WITH PASS STATEMENT – EMPTY FUNCTION PLACEHOLDER.

CODE:

```
x = 10
if x > 5:
    pass   # no action is performed
print("Program executed successfully")

#pass in a for loop
for i in range(1, 6):
    pass
print("Loop completed")

#pass in a while loop
i = 1
while i <= 5:
    pass
    i += 1
print(i)
print("While loop ended")

#pass in a function
def show_message():
    pass   # function body is empty
show_message()
print("Function executed")

#pass in a class
class Student:
    pass   # empty class
s = Student()
print("Class object created")
```

```
#pass inside a conditional loop
for i in range(5):
    if i == 3:
        pass
    else:
        print(i)
```

OUTPUT:



EX.NO: 14

PROGRAM WITH STRING INDEXING AND SLICING.

CODE:

```
text = "PYTHON"

#Basic Slicing
print(text[0:4])

#Slice from Start
print(text[:3])

#Slice till End
print(text[2:])

#Negative Index Slicing
print(text[-4:-1])

#Slicing with Step
print(text[::2])

#Reverse a String
print(text[::-1])

#Copy a String
new_text = text[:]
print(new_text)
```

OUTPUT:

Output

Clear

```
PYTH
PYT
THON
THO
PTO
NOHTYP
PYTHON

=== Code Execution Successful ===
```

EX.NO: 15

PROGRAM WITH STRING METHODS AND STRING VALIDATION.

CODE:

```
def string_methods_demo(text):
    print("\n--- String Methods Demo ---")
    print("Original String:", text)
    print("Uppercase:", text.upper())
    print("Lowercase:", text.lower())
    print("Title Case:", text.title())
    print("Length:", len(text))
    print("First 5 characters:", text[:5])
    print("Replace 'a' with '@':", text.replace('a', '@'))
    print("Split into words:", text.split())
    print("Starts with 'Hello':", text.startswith("Hello"))
    print("Ends with 'world':", text.endswith("world"))


def string_validation_demo(text):
    print("\n--- String Validation Demo ---")
    print("Is alphabetic?:", text.isalpha())
    print("Is numeric?:", text.isdigit())
    print("Is alphanumeric?:", text.isalnum())
    print("Is lowercase?:", text.islower())
    print("Is uppercase?:", text.isupper())
    print("Contains only spaces?:", text.isspace())


# Main Program
user_input = input("Enter a string: ")

string_methods_demo(user_input)
string_validation_demo(user_input)
```

OUTPUT:

```
Output                                    Clear
Enter a string: madiha
madiha

--- String Methods Demo ---
Original String: madiha
Uppercase: MADIHA
Lowercase: madiha
Title Case: Madiha
Length: 6
First 5 characters: madih
Replace 'a' with '@': m@dih@
Split into words: ['madiha']
Starts with 'Hello': False
Ends with 'world': False

--- String Validation Demo ---
Is alphabetic?: True
Is numeric?: False
Is alphanumeric?: True
Is lowercase?: True
Is uppercase?: False
Contains only spaces?: False

=== Code Execution Successful ===
```

EX.NO: 16

PROGRAM DEFINING AND CALLING A FUNCTION.

CODE:

```
# 1. Simple function (no parameters, no return value)
def greet():
    print("Hello! Welcome to Python programming.")

# 2. Function with parameters
def add(a, b):
    print("Sum =", a + b)

# 3. Function with return value
def multiply(a, b):
    return a * b

# 4. Function with default parameter
def power(base, exp=2):
    return base ** exp

# 5. Function with variable-length arguments (*args)
def total(*numbers):
    return sum(numbers)

# 6. Function with keyword arguments (**kwargs)
def student_info(**details):
    for key, value in details.items():
        print(key, ":", value)

# 7. Recursive function
def factorial(n):
    if n == 1:
        return 1
```

```
    return n * factorial(n - 1)

# FUNCTION CALLS
greet()                     # Calling simple function
add(10, 20)                 # Calling function with parameters
result = multiply(4, 5)     # Calling function with return value
print("Multiplication =", result)

print("Power =", power(5))      # Default parameter
print("Power =", power(5, 3))   # Passing argument

print("Total =", total(1, 2, 3, 4, 5))   # *args
print("Student Information:")
student_info(name="Madiha", age=19, course="Python")  # **kwargs
print("Factorial =", factorial(5))    # Recursive function
```

OUTPUT:

EX.NO: 17

PROGRAM WITH FUNCTION PARAMETERS.

CODE:

```
# 1. Positional Parameters
def positional_params(a, b):
    print("Positional Parameters:")
    print("a =", a)
    print("b =", b)
    print()

# 2. Keyword Parameters
def keyword_params(a, b):
    print("Keyword Parameters:")
    print("a =", a)
    print("b =", b)
    print()

# 3. Default Parameters
def default_params(a, b=10):
    print("Default Parameters:")
    print("a =", a)
    print("b =", b)
    print()

# 4. Variable-length Parameters (*args)
def variable_length_params(*args):
    print("Variable-length Parameters (*args):")
    for i in args:
        print(i, end=" ")
    print("\n")
```

```
# 5. Keyword Variable-length Parameters (**kwargs)
def keyword_variable_length_params(**kwargs):
    print("Keyword Variable-length Parameters (**kwargs):")
    for key, value in kwargs.items():
        print(key, "=", value)
    print()

# 6. Mixed Parameters
def mixed_params(a, b=5, *args, **kwargs):
    print("Mixed Parameters:")
    print("a =", a)
    print("b =", b)
    print("Additional positional arguments:", args)
    print("Additional keyword arguments:", kwargs)
    print()

# FUNCTION CALLS
positional_params(2, 3)
keyword_params(b=20, a=10)
default_params(5)
default_params(5, 15)
variable_length_params(1, 2, 3, 4, 5)
keyword_variable_length_params(name="madiha", age=19, course="Python")
mixed_params(10, 20, 30, 40, city="Delhi", country="India")
```

OUTPUT:

Output

```
b = 3

Keyword Parameters:
a = 10
b = 20

Default Parameters:
a = 5
b = 10

Default Parameters:
a = 5
b = 15

Variable-length Parameters (*args):
1 2 3 4 5

Keyword Variable-length Parameters (**kwargs):
name = madiha
age = 19
course = Python

Mixed Parameters:
a = 10
b = 20
Additional positional arguments: (30, 40)
Additional keyword arguments: {'city': 'Delhi', 'country': 'India'}


=== Code Execution Successful ===
```

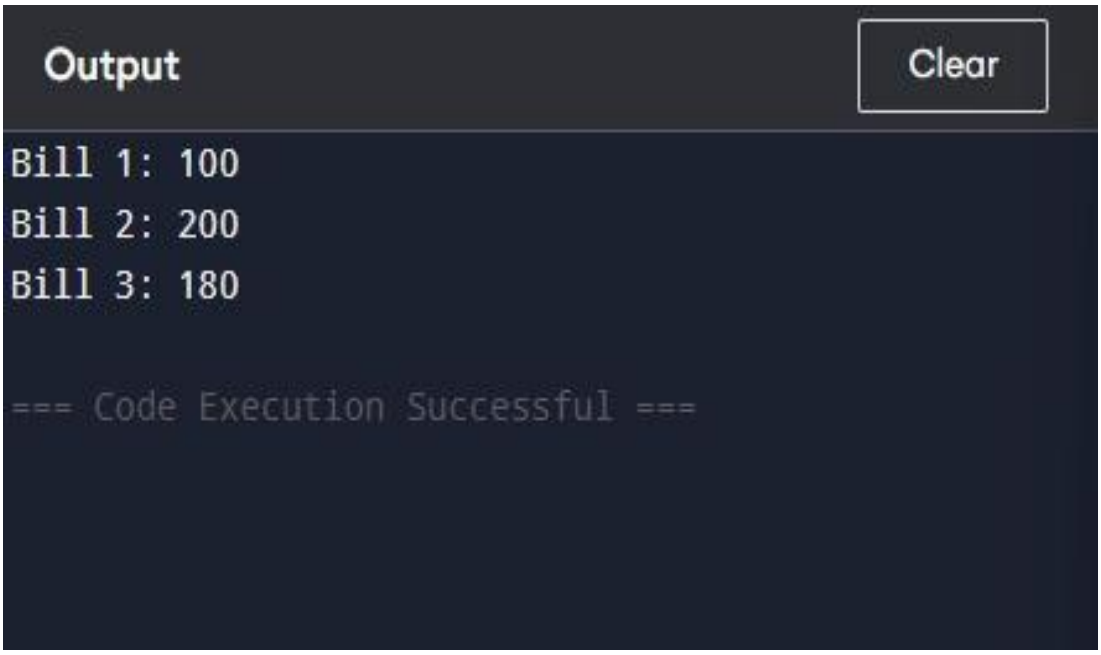
EX.NO: 18

Program with return values and default arguments.

CODE:

```
def calculate_total(price, quantity=1, discount=0):
 # Calculate total cost
    total = price * quantity
 # Apply discount
    total = total - discount
    return total   # return value
# Function calls
bill1 = calculate_total(100)              # uses default quantity and discount
bill2 = calculate_total(100, 2)            # default discount
bill3 = calculate_total(100, 2, 20)          # no default used
# Display results
print("Bill 1:", bill1)
print("Bill 2:", bill2)
print("Bill 3:", bill3)
```

OUTPUT:



EX.NO: 19

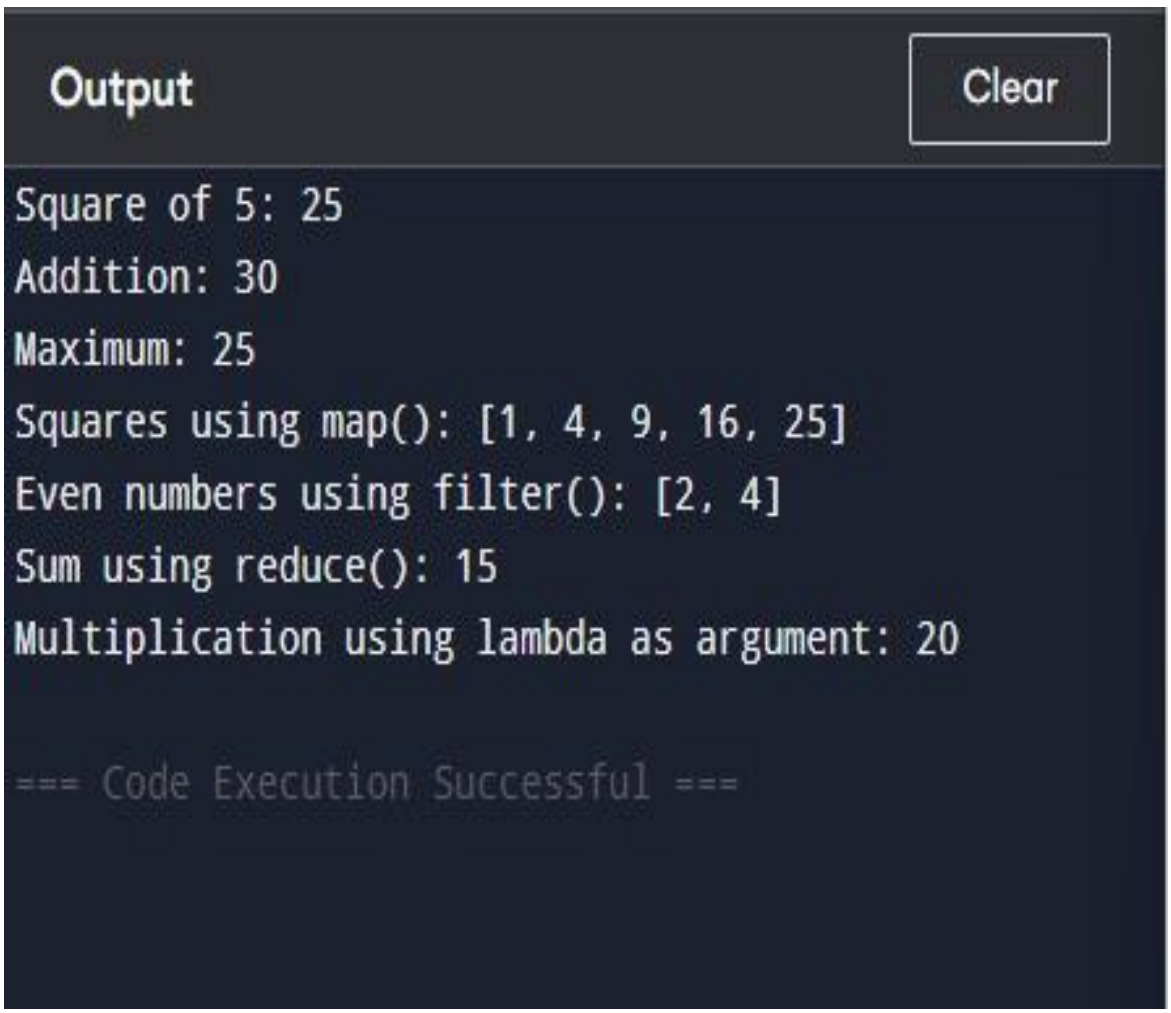
PROGRAM WITH LAMBDA FUNCTION.

CODE:

```
from functools import reduce

# 1. Simple lambda (single argument)
square = lambda x: x * x

# 2. Lambda with multiple arguments
add = lambda a, b: a + b

# 3. Lambda with conditional expression
maximum = lambda a, b: a if a > b else b

# 4. Lambda used with map()
numbers = [1, 2, 3, 4, 5]
squares_list = list(map(lambda x: x * x, numbers))

# 5. Lambda used with filter()
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))

# 6. Lambda used with reduce()
sum_of_numbers = reduce(lambda x, y: x + y, numbers)

# 7. Lambda as an argument to a function
def operate(func, a, b):
    return func(a, b)

result_operation = operate(lambda x, y: x * y, 4, 5)

# Display outputs
```

```
print("Square of 5:", square(5))
print("Addition:", add(10, 20))
print("Maximum:", maximum(15, 25))
print("Squares using map():", squares_list)
print("Even numbers using filter():", even_numbers)
print("Sum using reduce():", sum_of_numbers)
print("Multiplication using lambda as argument:", result_operation)
```

OUTPUT:



EX.NO: 20

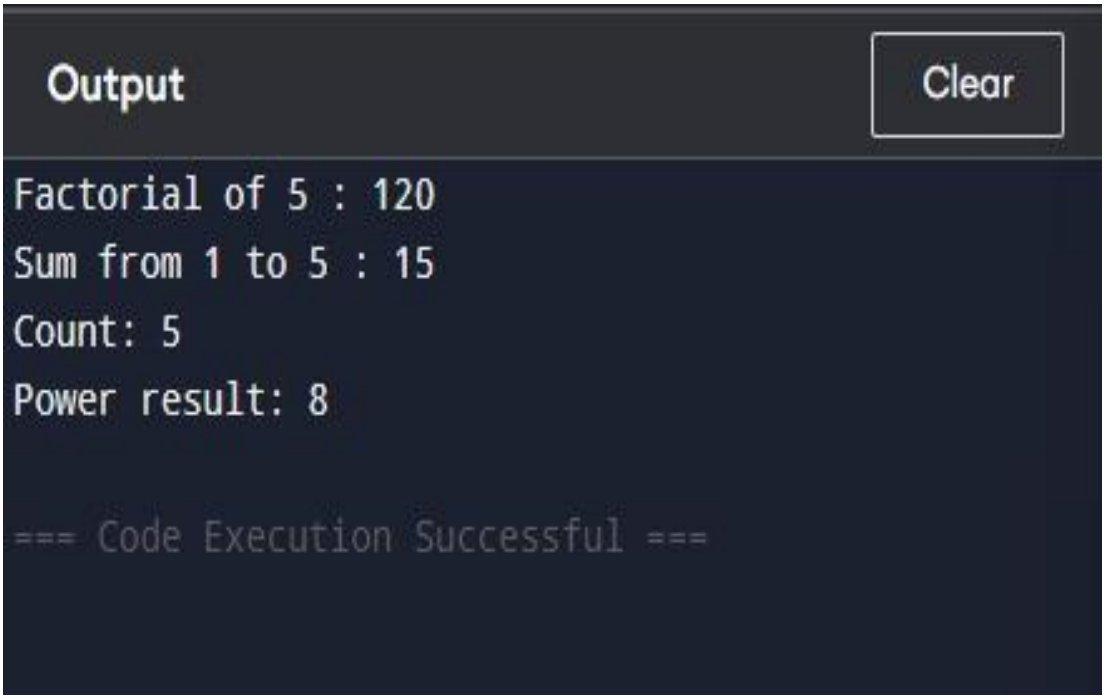
PROGRAM WITH RECURSION.

CODE:

```
def factorial(n):
    """
    This function calculates factorial of a number using recursion
    """

# 1. Base condition
    if n == 0 or n == 1:
        return 1
# 2. Recursive call
    else:
        return n * factorial(n - 1)


# Recursive function with multiple return values
def sum_and_count(n):
# Base condition
    if n == 0:
        return 0, 0
# Recursive call
    total, count = sum_and_count(n - 1)
    return total + n, count + 1


# Recursive function with default argument
def power(base, exp=1):
 # Base condition
    if exp == 0:
        return 1
```

```
# Recursive call
    return base * power(base, exp - 1)

# Function call section
num = 5
fact_result = factorial(num)
sum_result, count_result = sum_and_count(num)
power_result = power(2, 3)

# Display outputs
print("Factorial of", num, ":", fact_result)
print("Sum from 1 to", num, ":", sum_result)
print("Count:", count_result)
print("Power result:", power_result)
```

OUTPUT:



EX.NO: 21

PROGRAM TO DEMONSTRATE SCOPE – LOCAL VS GLOBAL VARIABLE.

CODE:

```
x = 10   # Global variable
def show_global():
    # Accessing global variable (read-only)
    print("Inside show_global(), x =", x)
def modify_global():
    # Modifying global variable using 'global' keyword
    global x
    x = 20
    print("Inside modify_global(), x modified to", x)
def local_scope_example():
    # LOCAL SCOPE
    y = 5   # Local variable
    print("Inside local_scope_example(), y =", y)
def shadowing_example():
    # Local variable shadows the global variable x
    x = 100
    print("Inside shadowing_example(), local x =", x)
def parameter_scope(a):
    # Function parameter is local to the function
    print("Inside parameter_scope(), a =", a)
def nested_function_example():
    # ENCLOSING (NONLOCAL) SCOPE
    z = 50  # Enclosing variable
 def inner_function():
        nonlocal z
        z = 60
print("Inside inner_function(), z =", z)
print("Before calling inner_function(), z =", z)
```

```
    inner_function()
    print("After calling inner_function(), z =", z)
# MAIN PROGRAM EXECUTION
print("Global x initially =", x)
show_global()
local_scope_example()
shadowing_example()
print("After shadowing_example(), global x still =", x)
modify_global()
print("After modify_global(), global x =", x)
parameter_scope(25)
nested_function_example()
```

OUTPUT:

```
Output                                                    Clear
Global x initially = 10
Inside show_global(), x = 10
Inside local_scope_example(), y = 5
Inside shadowing_example(), local x = 100
After shadowing_example(), global x still = 10
Inside modify_global(), x modified to 20
After modify_global(), global x = 20
Inside parameter_scope(), a = 25
Before calling inner_function(), z = 50
Inside inner_function(), z = 60
After calling inner_function(), z = 60

=== Code Execution Successful ===
```

EX.NO: 22

PROGRAM WITH LIST OPERATIONS.

CODE:

```
#Step 1: Create lists
A = [1, 2, 3]
B = [4, 5]
print("List A:", A)
print("List B:", B)

# Step 2: Concatenation (+)
C = A + B
print("Concatenation (A + B):", C)

# Step 3: Repetition (*)
D = A * 3
print("Repetition (A * 3):", D)

# Step 4: Membership operators (in / not in)
print("Is 2 in A?", 2 in A)
print("Is 10 not in A?", 10 not in A)

# Step 5: Indexing operator
print("First element of A:", A[0])
print("Last element of A:", A[-1])

# Step 6: Slicing operator
print("Slice A[0:2]:", A[0:2])
print("Slice A[:2]:", A[:2])
print("Slice A[1:]:", A[1:])

# Step 7: Assignment operator
```

```
A[1] = 20
print("After assignment A[1] = 20:", A)

# Step 8: Comparison operators
X = [1, 2, 3]
Y = [1, 2, 4]

print("X == Y:", X == Y)
print("X < Y:", X < Y)
print("X > Y:", X > Y)

# Step 9: Length operator
print("Length of A:", len(A))

# Step 10: Looping using list
print("Elements of list A:")
for item in A:
    print(item)
```

OUTPUT:

Output

Clear

```
List A: [1, 2, 3]
List B: [4, 5]
Concatenation (A + B): [1, 2, 3, 4, 5]
Repetition (A * 3): [1, 2, 3, 1, 2, 3, 1, 2, 3]
Is 2 in A? True
Is 10 not in A? True
First element of A: 1
Last element of A: 3
Slice A[0:2]: [1, 2]
Slice A[:2]: [1, 2]
Slice A[1:]: [2, 3]
After assignment A[1] = 20: [1, 20, 3]
X == Y: False
X < Y: True
X > Y: False
Length of A: 3
Elements of list A:
1
20
3

=== Code Execution Successful ===
```

EX.NO: 23

PROGRAM WITH LIST COMPREHENSION.

CODE:

```
# Step 1: Original list
numbers = [1, 2, 3, 4, 5, 6]
print("Original list:", numbers)

# Step 2: Basic list comprehension
squares = [x**2 for x in numbers]
print("Squares:", squares)

# Step 3: List comprehension with condition (if)
evens = [x for x in numbers if x % 2 == 0]
print("Even numbers:", evens)

# Step 4: List comprehension with if-else
even_odd = ["Even" if x % 2 == 0 else "Odd" for x in numbers]
print("Even or Odd:", even_odd)

# Step 5: List comprehension with expression
add_ten = [x + 10 for x in numbers]
print("Add 10:", add_ten)

# Step 6: List comprehension using string
word = "python"
vowels = [ch for ch in word if ch in "aeiou"]
print("Vowels in word:", vowels)

# Step 7: Nested list comprehension
matrix = [[1, 2], [3, 4], [5, 6]]
flatten = [num for row in matrix for num in row]
```

```
print("Flattened list:", flatten)

# Step 8: List comprehension with range()
cubes = [x**3 for x in range(1, 6)]
print("Cubes:", cubes)
```

OUTPUT:

```
Output                                    Clear
Original list: [1, 2, 3, 4, 5, 6]
Squares: [1, 4, 9, 16, 25, 36]
Even numbers: [2, 4, 6]
Even or Odd: ['Odd', 'Even', 'Odd', 'Even', 'Odd',
    'Even']
Add 10: [11, 12, 13, 14, 15, 16]
Vowels in word: ['o']
Flattened list: [1, 2, 3, 4, 5, 6]
Cubes: [1, 8, 27, 64, 125]

=== Code Execution Successful ===
```

EX.NO: 24

PROGRAM WITH TUPLE.

CODE:

```
# Step 1: Creating a tuple
student = ("Madiha", 20, "BSc", 85, 90, 85)

# Step 2: Display the tuple
print("Tuple:", student)

# Step 3: Type of tuple
print("Type:", type(student))

# Step 4: Length of tuple
print("Length:", len(student))

# Step 5: Accessing elements
print("First element:", student[0])
print("Last element:", student[-1])

# Step 6: Slicing
print("Slice (1:4):", student[1:4])

# Step 7: Immutability
# student[1] = 21   # Error: Tuple is immutable

# Step 8: Tuple concatenation
extra = ("Passed",)
new_student = student + extra
print("After concatenation:", new_student)

# Step 9: Tuple repetition
```

```
print("Repetition:", extra * 3)

# Step 10: Tuple membership
print("Is 'madiha' in tuple?", "Madiha" in student)

# Step 11: Tuple methods
print("Count of 85:", student.count(85))
print("Index of 'BSc':", student.index("BSc"))

# Step 12: Tuple unpacking
name, age, course, m1, m2, m3 = student
print("Unpacked Name:", name)
print("Unpacked Age:", age)

# Step 13: Looping through tuple
print("Elements in tuple:")
for item in student:
    print(item)

# Step 14: Nested tuple
nested = (student, ("Python", "Maths"))
print("Nested Tuple:", nested)
```

OUTPUT:

Output

Clear

```
Tuple: ('Madiha', 20, 'BSc', 85, 90, 85)
Type: <class 'tuple'>
Length: 6
First element: Madiha
Last element: 85
Slice (1:4): (20, 'BSc', 85)
After concatenation: ('Madiha', 20, 'BSc', 85, 90,
    85, 'Passed')
Repetition: ('Passed', 'Passed', 'Passed')
Is 'madiha' in tuple? True
Count of 85: 2
Index of 'BSc': 2
Unpacked Name: Madiha
Unpacked Age: 20
Elements in tuple:
Madiha
20
BSc
85
90
85
Nested Tuple: (('Madiha', 20, 'BSc', 85, 90, 85),
    ('Python', 'Maths'))

=== Code Execution Successful ===
```

EX.NO: 25

PROGRAM WITH SET.

CODE:

```
# Step 1: Creating sets
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7}

# Step 2: Display sets
print("Set A:", A)
print("Set B:", B)

# Step 3: Type and length
print("Type of A:", type(A))
print("Length of A:", len(A))

# Step 4: Unordered and unique elements
C = {1, 2, 2, 3, 3, 4}
print("Set with duplicates removed:", C)

# Step 5: Add elements
A.add(10)
print("After add():", A)

# Step 6: Update set
A.update([11, 12])
print("After update():", A)

# Step 7: Remove elements
A.remove(3)       # Removes element 3
A.discard(100)    # No error if element not present
print("After remove & discard:", A)
```

```
# Step 8: Pop an element
A.pop()
print("After pop():", A)

# Step 9: Set membership
print("Is 4 in A?", 4 in A)

# Step 10: Set operations
print("Union:", A | B)
print("Intersection:", A & B)
print("Difference (A - B):", A - B)
print("Symmetric Difference:", A ^ B)

# Step 11: Set methods
print("Is A subset of B?", A.issubset(B))
print("Is A superset of B?", A.issuperset(B))
print("Are A and B disjoint?", A.isdisjoint(B))

# Step 12: Looping through a set
print("Elements of A:")
for item in A:
    print(item)

# Step 13: Frozen set (immutable set)
F = frozenset([1, 2, 3])
print("Frozen Set:", F)
```

OUTPUT:

```
Output                                                    Clear
Set A: {1, 2, 3, 4, 5}
Set B: {4, 5, 6, 7}
Type of A: <class 'set'>
Length of A: 5
Set with duplicates removed: {1, 2, 3, 4}
After add(): {1, 2, 3, 4, 5, 10}
After update(): {1, 2, 3, 4, 5, 10, 11, 12}
After remove & discard: {1, 2, 4, 5, 10, 11, 12}
After pop(): {2, 4, 5, 10, 11, 12}
Is 4 in A? True
Union: {2, 4, 5, 6, 7, 10, 11, 12}
Intersection: {4, 5}
Difference (A - B): {11, 2, 10, 12}
Symmetric Difference: {2, 6, 7, 10, 11, 12}
Is A subset of B? False
Is A superset of B? False
Are A and B disjoint? False
Elements of A:
2
4
5
10
11
12
Frozen Set: frozenset({1, 2, 3})

=== Code Execution Successful ===
```

EX.NO: 26

PROGRAM WITH DICTIONARY.

CODE:

```
# Step 1: Creating a dictionary
student = {
    "name": "Madiha",
    "age": 20,
    "course": "BSc",
    "marks": [85, 90, 88]
}
# Step 2: Display dictionary
print("Dictionary:", student)
# Step 3: Type and length
print("Type:", type(student))
print("Length:", len(student))
# Step 4: Accessing values using keys
print("Name:", student["name"])
print("Age:", student.get("age"))
# Step 5: Adding new key-value pair
student["grade"] = "A"
print("After adding grade:", student)
# Step 6: Updating value
student["age"] = 21
print("After updating age:", student)
# Step 7: Dictionary is mutable
# Values can be changed but keys must be unique
# Step 8: Removing items
student.pop("course")        # Remove specific key
removed = student.popitem() # Remove last inserted item
print("After pop & popitem:", student)
# Step 9: Keys, Values, Items
```

```
print("Keys:", student.keys())
print("Values:", student.values())
print("Items:", student.items())
# Step 10: Membership testing
print("Is 'name' a key?", "name" in student)
# Step 11: Looping through dictionary
print("Looping through dictionary:")
for key, value in student.items():
    print(key, ":", value)
# Step 12: Copying dictionary
student_copy = student.copy()
print("Copied dictionary:", student_copy)
# Step 13: Nested dictionary
college = {
    "student1": student,
    "student2": {
        "name": "Madiha",
        "age": 22,
        "course": "BCom"
    }
}
print("Nested Dictionary:", college)
# Step 14: Clearing dictionary
student_copy.clear()
print("After clear():", student_copy)
```

OUTPUT:

```
Output                                                    Clear
Dictionary: {'name': 'Madiha', 'age': 20, 'course': 'BSc', 'marks': [85, 90, 88]}
Type: <class 'dict'>
Length: 4
Name: Madiha
Age: 20
After adding grade: {'name': 'Madiha', 'age': 20, 'course': 'BSc', 'marks': [85, 90, 88], 'grade':
    'A'}
After updating age: {'name': 'Madiha', 'age': 21, 'course': 'BSc', 'marks': [85, 90, 88], 'grade':
    'A'}
After pop & popitem: {'name': 'Madiha', 'age': 21, 'marks': [85, 90, 88]}
Keys: dict_keys(['name', 'age', 'marks'])
Values: dict_values(['Madiha', 21, [85, 90, 88]])
Items: dict_items([('name', 'Madiha'), ('age', 21), ('marks', [85, 90, 88])])
Is 'name' a key? True
Looping through dictionary:
name : Madiha
age : 21
marks : [85, 90, 88]
Copied dictionary: {'name': 'Madiha', 'age': 21, 'marks': [85, 90, 88]}
Nested Dictionary: {'student1': {'name': 'Madiha', 'age': 21, 'marks': [85, 90, 88]}, 'student2':
    {'name': 'Madiha', 'age': 22, 'course': 'BCom'}}
After clear(): {}

=== Code Execution Successful ===
```

EX.NO: 27

PROGRAM WITH ARRAY.

CODE:

```
# Step 1: Import array module
from array import array

# Step 2: Create an array of integers
arr = array('i', [10, 20, 30, 40, 50])

# Step 3: Display array
print("Array:", arr)

# Step 4: Type of array
print("Type:", type(arr))

# Step 5: Length of array
print("Length:", len(arr))

# Step 6: Access elements using index
print("First element:", arr[0])
print("Last element:", arr[-1])

# Step 7: Slicing
print("Slice (1:4):", arr[1:4])

# Step 8: Modify elements (arrays are mutable)
arr[2] = 35
print("After modification:", arr)

# Step 9: Append element
```

```
arr.append(60)
print("After append:", arr)

# Step 10: Insert element
arr.insert(1, 15)
print("After insert:", arr)

# Step 11: Remove element
arr.remove(40)
print("After remove:", arr)

# Step 12: Pop element
arr.pop()
print("After pop:", arr)

# Step 13: Index and count
print("Index of 20:", arr.index(20))
print("Count of 10:", arr.count(10))

# Step 14: Loop through array
print("Elements in array:")
for item in arr:
    print(item)

# Step 15: Convert array to list
lst = arr.tolist()
print("Converted to list:", lst)
```

OUTPUT:

```
Output                                                    Clear
Array: array('i', [10, 20, 30, 40, 50])
Type: <class 'array.array'>
Length: 5
First element: 10
Last element: 50
Slice (1:4): array('i', [20, 30, 40])
After modification: array('i', [10, 20, 35, 40, 50]
    )
After append: array('i', [10, 20, 35, 40, 50, 60])
After insert: array('i', [10, 15, 20, 35, 40, 50,
    60])
After remove: array('i', [10, 15, 20, 35, 50, 60])
After pop: array('i', [10, 15, 20, 35, 50])
Index of 20: 2
Count of 10: 1
Elements in array:
10
15
20
35
50
Converted to list: [10, 15, 20, 35, 50]

=== Code Execution Successful ===
```

EX.NO: 28

PROGRAM WITH TRY–EXCEPT–FINALLY – HANDLE MULTIPLE EXCEPTIONS.

CODE:

```
try:
    x = int(input("Enter first number: "))
    y = int(input("Enter second number: "))

    lst = [10, 20, 30]

    print("Division:", x / y)
    print("List value:", lst[2])   # fixed index

except ValueError:
    print("ValueError: Invalid integer input")

except ZeroDivisionError:
    print("ZeroDivisionError: Cannot divide by zero")

except IndexError:
    print("IndexError: List index out of range")

except Exception as e:
    print("Other error:", e)

finally:
    print("Execution finished (finally block always runs)")
```

OUTPUT:

main.py Output

```
Enter first number: 100
Enter second number: 10
Division: 10.0
List value: 30
Execution finished (finally block always runs)

=== Code Execution Successful ===
```

EX.NO: 29

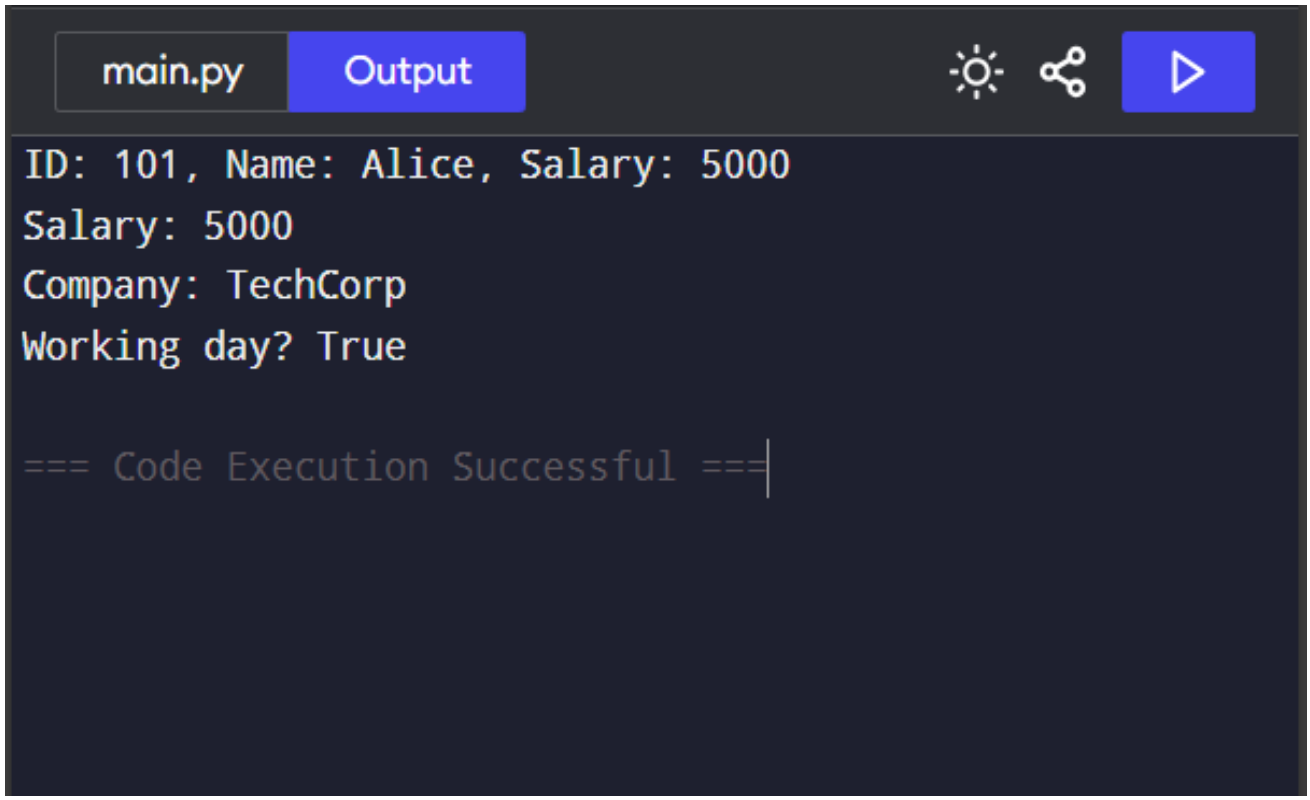
PROGRAM WITH CLASS AND OBJECT.

CODE:

```
class Person:
    company = "TechCorp"          # Class variable
    def __init__(self, name, salary):
        self.name = name          # Instance variable
        self.__salary = salary    # Private variable (Encapsulation)



    # Instance method
    def show(self):
        return f"Name: {self.name}, Salary: {self.__salary}"
    # Getter (Encapsulation)
    def get_salary(self):
        return self.__salary
    # Class method
    @classmethod
    def get_company(cls):
        return cls.company
    # Static method
    @staticmethod
    def is_working_day(day):
        return day.lower() not in ["saturday", "sunday"]
    # Magic method
    def __str__(self):
        return self.show()
# Inheritance & Polymorphism
class Employee(Person):
    def __init__(self, name, salary, emp_id):
        super().__init__(name, salary)
```

```
        self.emp_id = emp_id
    def show(self):   # Method overriding
        return f"ID: {self.emp_id}, " + super().show()
# Object creation
emp = Employee("Alice", 5000, 101)

print(emp)                          # __str__()
print("Salary:", emp.get_salary())      # Encapsulation
print("Company:", Employee.get_company())# Class method
print("Working day?", Employee.is_working_day("Monday"))
```

OUTPUT:



EX.NO: 30

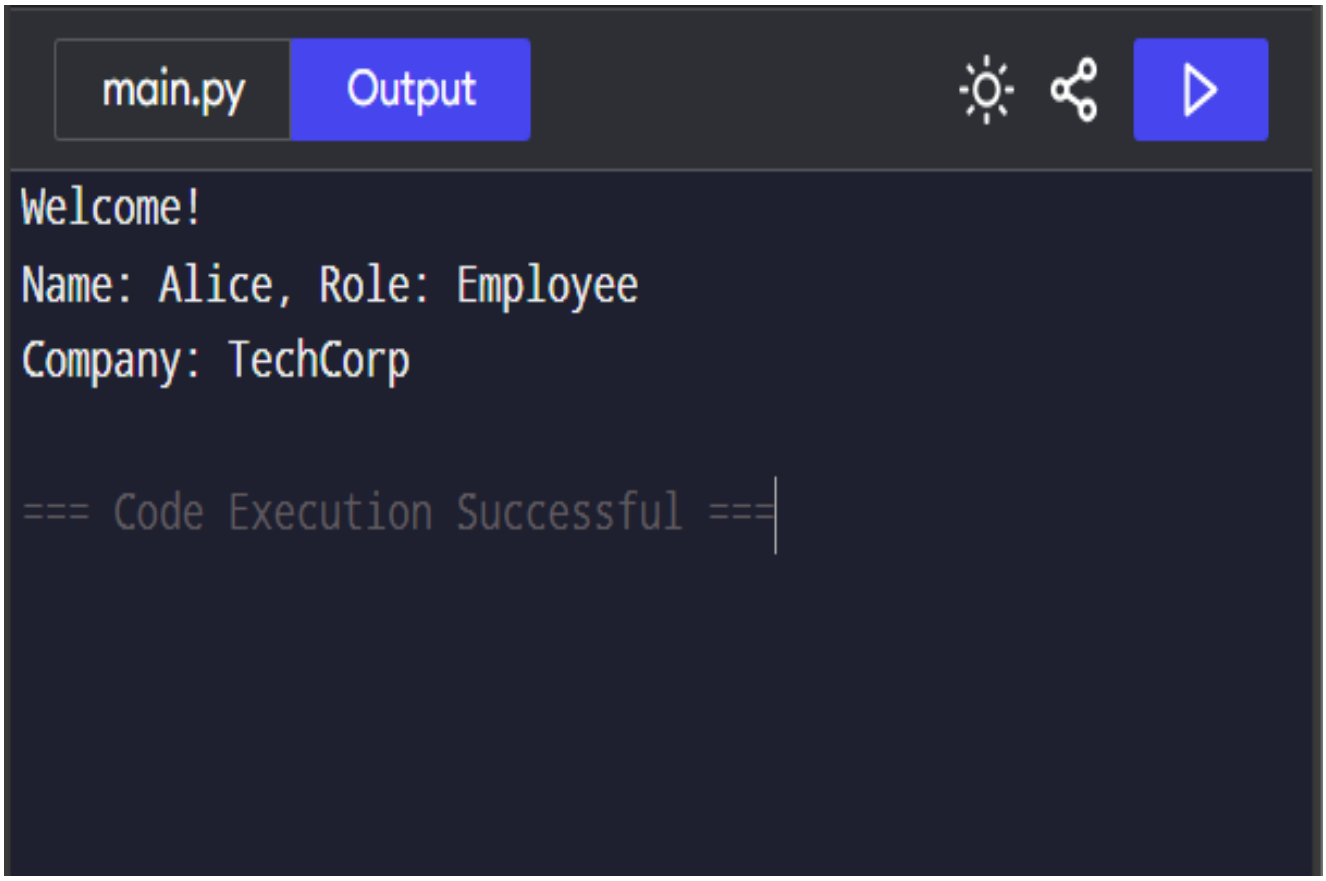
PROGRAM WITH INHERITANCE.

CODE:

```
class Person:
    company = "TechCorp"          # Class variable
    def __init__(self, name):
        self.name = name          # Instance variable
    def role(self):
        return "Person"
    @classmethod
    def get_company(cls):
        return cls.company
    @staticmethod
    def greet():
        return "Welcome!"
    def __str__(self):
        return f"Name: {self.name}, Role: {self.role()}"

# Inheritance + Polymorphism
class Employee(Person):
    def __init__(self, name, emp_id):
        super().__init__(name)
        self.emp_id = emp_id

    def role(self):               # Method overriding
        return "Employee"
# Object creation
emp = Employee("Alice", 101)

print(Person.greet())             # Static method
print(emp)                        # __str__ + Polymorphism
```

```
print("Company:", emp.get_company())
```

OUTPUT:



EX.NO: 31

PROGRAM WITH ENCAPSULATION – GETTERS/SETTERS.

CODE:

```
class Employee:
    company = "TechCorp"          # Class variable
    def __init__(self, name, salary):
        self.name = name          # Public
        self.__salary = salary    # Private (Encapsulation)
    # Getter
    def get_salary(self):
        return self.__salary
    # Setter
    def set_salary(self, salary):
        if salary > 0:
            self.__salary = salary
    @classmethod
    def get_company(cls):
        return cls.company
    @staticmethod
    def info():
        return "Employee Details"
    def __str__(self):
        return f"Name: {self.name}, Salary: {self.__salary}"

# Object creation
emp = Employee("Alice", 5000)
emp.set_salary(6000)              # Setter
print(Employee.info())            # Static method
print(emp)                        # Magic method
print("Company:", emp.get_company())
```

OUTPUT:

```
main.py    Output

Employee Details
Name: Alice, Salary: 6000
Company: TechCorp

=== Code Execution Successful ===
```

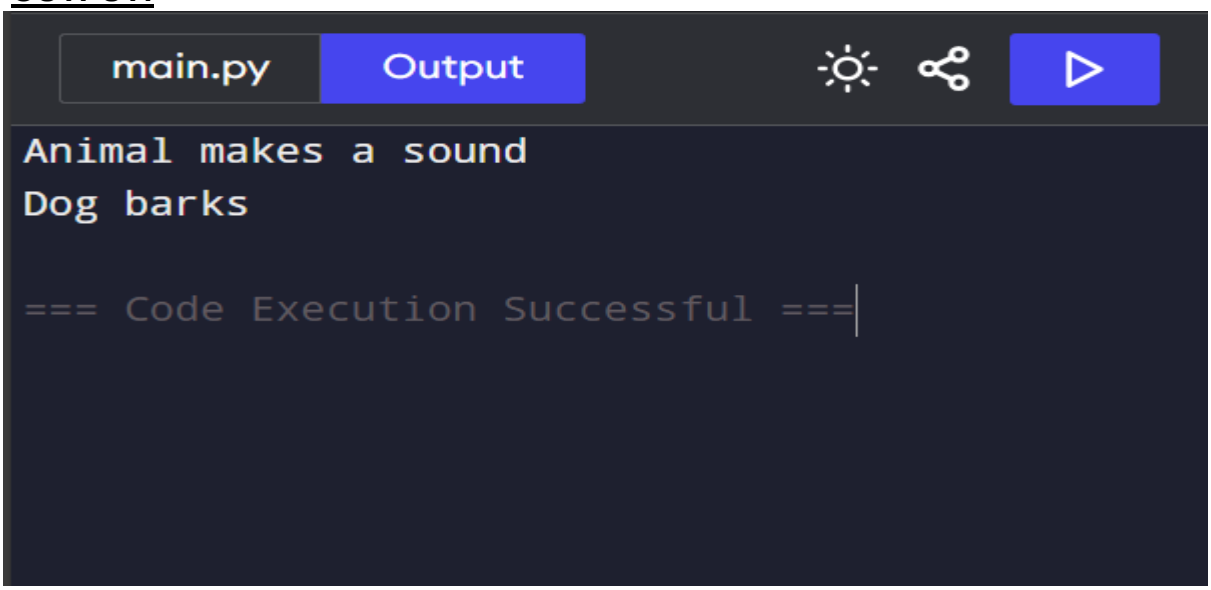
EX.NO: 32

PROGRAM WITH POLYMORPHISM – METHOD OVERRIDING.

CODE:

```
class Animal:
    def sound(self):
        return "Animal makes a sound"
    def __str__(self):
        return self.sound()

# Polymorphism: Method Overriding
class Dog(Animal):
    def sound(self):
        return "Dog barks"

# Object creation
a = Animal()
d = Dog()
print(a)    # Animal method
print(d)    # Overridden method
```

OUTPUT:



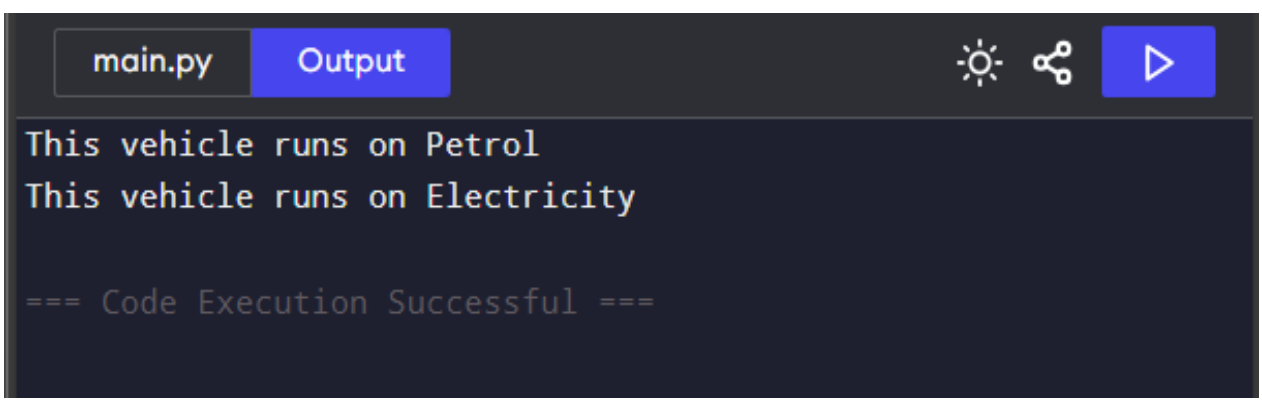
EX.NO: 33

PROGRAM WITH ABSTRACTION – ABSTRACT CLASS.

CODE:

```
from abc import ABC, abstractmethod
# Abstract Class
class Vehicle(ABC):
    @abstractmethod
    def fuel_type(self):
        pass
    def __str__(self):
        return f"This vehicle runs on {self.fuel_type()}"
# Derived class implements abstract method
class Car(Vehicle):
    def fuel_type(self):          # Method implementation
        return "Petrol"
class ElectricCar(Vehicle):
    def fuel_type(self):
        return "Electricity"
# Objects
v1 = Car()
v2 = ElectricCar()
print(v1)  # Magic method + abstraction
print(v2)
```

OUTPUT:



EX.NO: 34

PROGRAM USING MODULES – IMPORT MATH & USE TRIGONOMETRIC FUNCTIONS.

CODE:

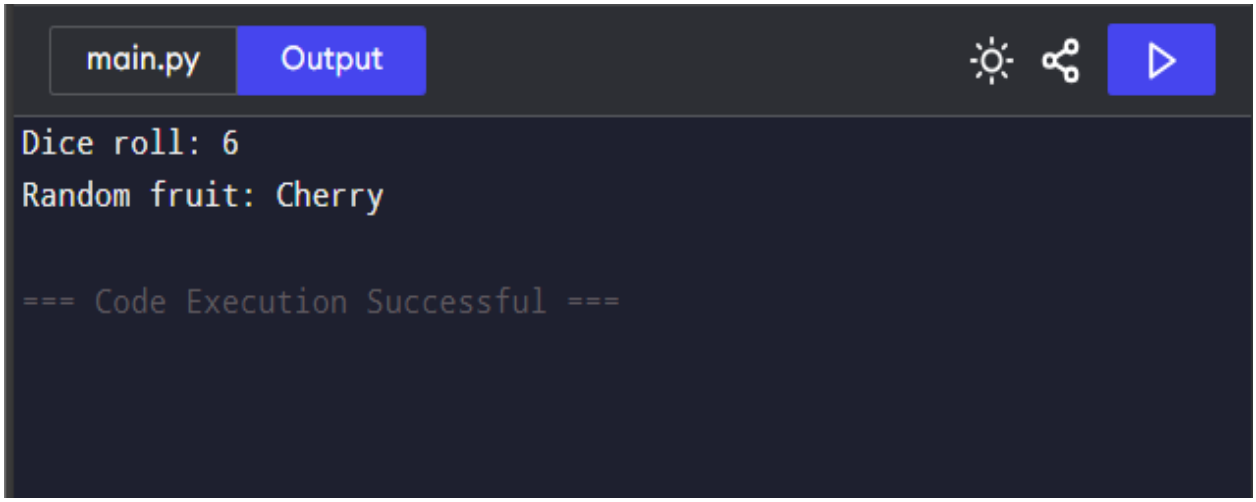
```
import math  # Importing module

# Angle in degrees
angle_deg = 30

# Convert degrees to radians
angle_rad = math.radians(angle_deg)

# Trigonometric functions
sin_val = math.sin(angle_rad)
cos_val = math.cos(angle_rad)
tan_val = math.tan(angle_rad)

# Output
print(f"Angle: {angle_deg}°")
print(f"Sin: {sin_val:.2f}, Cos: {cos_val:.2f}, Tan: {tan_val:.2f}")
```

OUTPUT:

EX.NO: 35

PROGRAM USING RANDOM MODULE.

CODE:

```
import random  # Import random module
class Dice:
    def __init__(self, sides=6):
        self.sides = sides  # Encapsulation could be added with __sides
    def roll(self):
        return random.randint(1, self.sides)  # Random integer
    @staticmethod
    def random_choice(options):
        return random.choice(options)  # Random selection from list

# Object creation
dice = Dice()
# Rolling the dice
print("Dice roll:", dice.roll())
# Random choice from list
fruits = ["Apple", "Banana", "Cherry"]
print("Random fruit:", Dice.random_choice(fruits))
```

OUTPUT: